Day 1 - Recap

- ✓ GenAI — FM / LLM (Models)
  - ↳ Github 'Copilot' → "AI Coding Assistant"
- ✓ Core Copilot use cases
  - ① Code Generation
  - ② Code explanation
  - ③ Test Generation
  - ④ Doc string / Readme.MD
  - ⑤ Refactoring the code
  - ⑥ Debugging the API/code
  - ⑦ Code Review ✓
- ✓ Prompt Engineering → 'Golden Template'
  - ① Role/Persona ② Rules ③ Examples
  - ④ Edge cases ⑤ Goal ⑥ output

Day 2 - Agenda

① MCP → Extending Copilot with MCP. ✓

② Copilot for Team Productivity
- ✓ Code Review
- ✓ Pull Requests

③ Governance (Readme.MD & Copilot-instructions.md)

④ [Cloud Agent / Coding Agent
- VS Code ⇒ Github Copilot chat < Agent / Ask / Plan]

⑤ Custom Agent — Multi Agents in Github Copilot

what is "MCP" (Model Context Protocol) ?

✓ Outside Context / Content / Knowledge Base → Github Copilot ✓ → (Model)

✓ MCP is a standard way for an AI Assistant to talk / connect to external tools & data sources

why it matters? (The Problem it solves)

✓ Copilot — strong at generating & explaining code.

✓ But it lacks,
- → Actual Repo Structure / modules (Github Repo / Azure Devops)
- → Current issue requirements (JIRA)
- → Latest decision (Confluence)
- → Real PR comment / diff context

MCP → fixes the "Missing context"

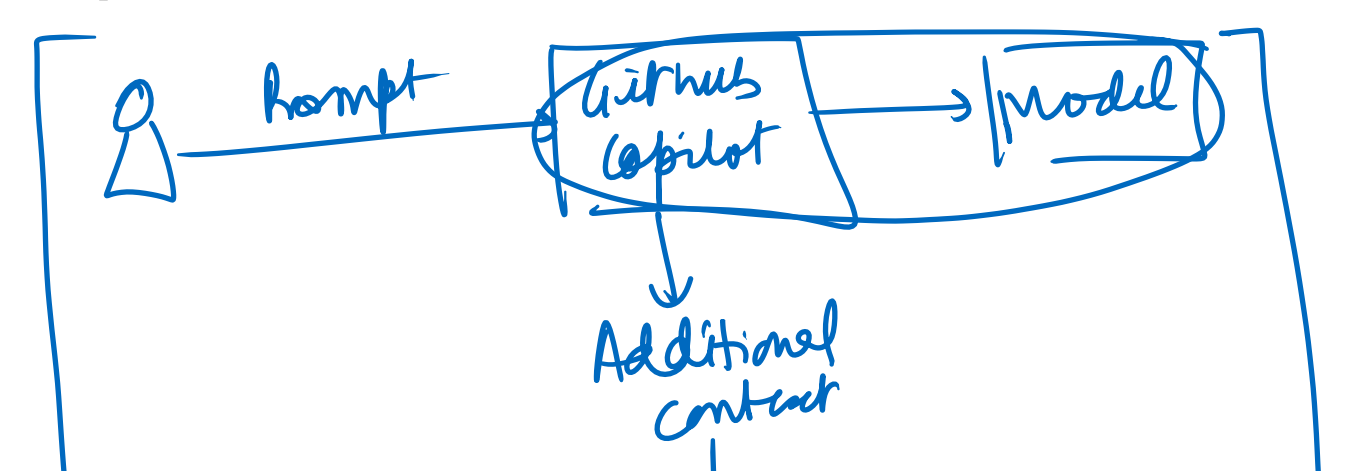
Prompting → Talking to a smart intern / Jr. Dev with only what you told them verbally.

MCP → Giving that Intern / Jr. Dev Read-only Access to your approved systems (Repo, issues, docs) logs all the request.

Q How MCP enables Copilot to access external, structured context?

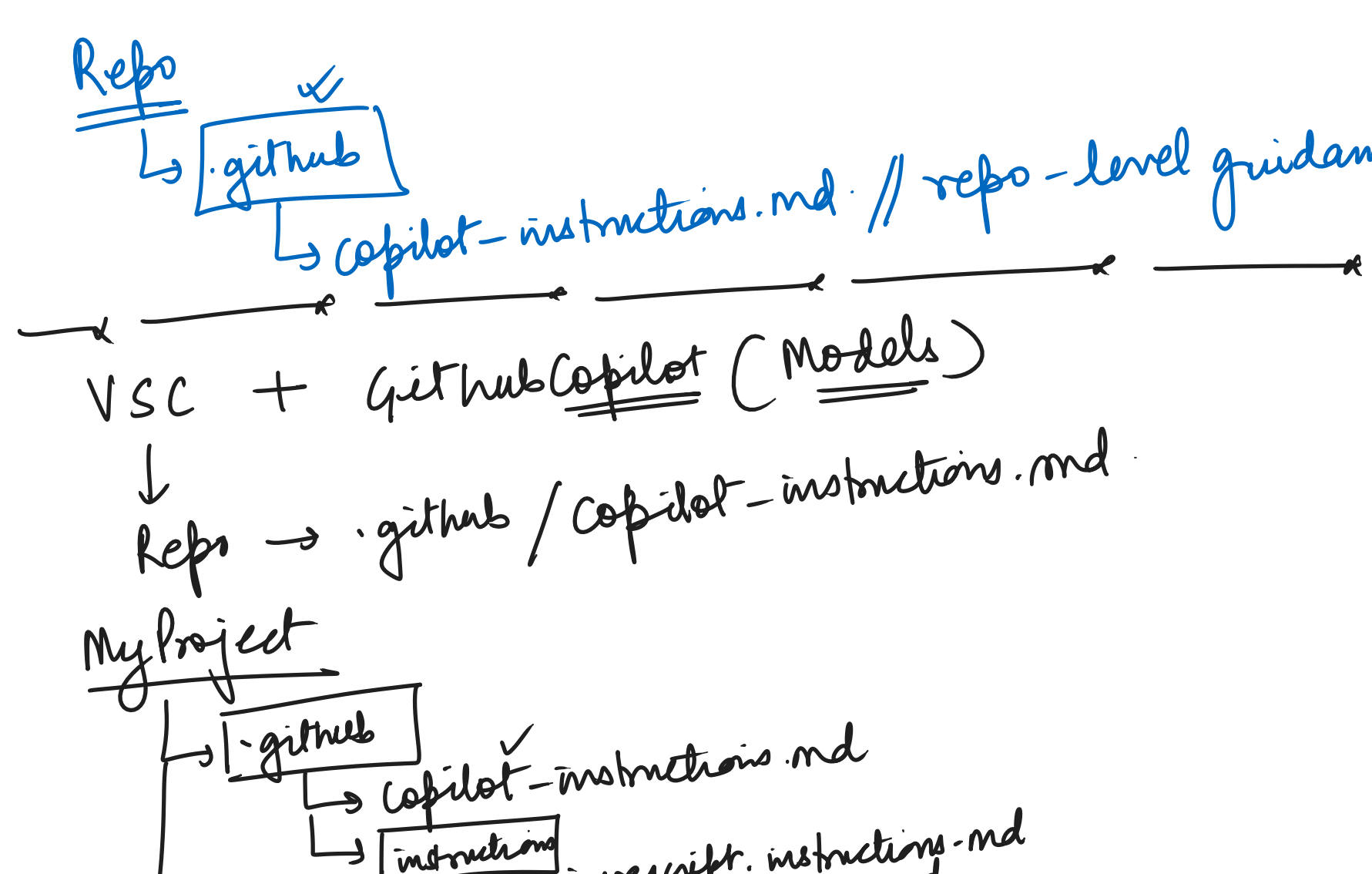
CORE IDEA: MCP ✓ Server < Public → (Github MCP Server) / Private →
- ↓ Expose Tools → (Copilot ⇒)
- ① searchIssues (query)
- ② getPullRequest (prNumber)

Github ✓ Copilot → MCP
- ① You ask a Qs.
- ② Copilot detects it needs external facts
- ③ It call MCP tool(s) to fetch Info (Structured Context)
- ④ It uses the info to pass to the Model, produce a better response



what is "Structured Context"?
- fields → status, priority
- objects → PRs, issues, pages, files
- Relationship : issue ↔ PRs, commit ↔ Repo

MCP Server
- ① Github MCP Server



Repo
- ↳ .github ✓
  - ↳ copilot-instructions.md // repo-level guidance

VSC + GithubCopilot (Models)

↓

Repo → .github / copilot-instructions.md

MyProject
- ↳ .github
  - ↳ copilot-instructions.md ✓
  - ↳ instructions
    - javascript.instructions.md
    - Tests.instructions.md
    - api.instructions.md
- ↳ /src/
- ↳ Tests/
- ↳ package.json
- ↳ Readme.md

Lab + Demo

Client → Express Router → Controller → Product Service → In-Memory Product Store

X Route
- → validation
- → Business logic
- → database logic
- → Secrets
- → error handling

Github Copilot

Capability	"Coding Agent" Copilot Agent in IDE (work with me)	vs.	Cloud Agent Copilot Cloud Agent (Repo) (Do this for me)
✓ where it runs?	Your local IDE / env		Github Hosted env.
✓ working style	Interactive / synch		Delegated / autonomous (Goal)
✓ Typical Tools	VS Code, VS, JetBrains		Github.com, Issues, Agents, PR workflow
✓ Code changes	Directly in working copy		Separate Branch
✓ Pull Request	You normally create/manage it.		Agent can create a draft PR.
"Can continue without actively coding	NO		Yes
✓ Best for	Debugging, implementing while pairing with AI		well-scoped Backlog Tasks.
✓ Env	Your Machine		Github-Actions powered Runner
✓ Review Model	Developer watches changes interactively		Developer reviews resulting PR.

Dev → IDE → Coding Agent → local workspace

Dev → Issue / Prompt → Cloud Agent → Github Runner → Branch → Tests → PR → Human Review

WORKFLOWS

① Github Issue

↓

Assign task to Copilot → Copilot / Claude / OpenAI

↓

Copilot Cloud Agent

↓

Create isolated development env
- — Inspect Repo
- — understand Instructions
- — Modify files
- — Build application
- — Run tests
- — Validate changes

↓

Create branch

↓

Draft PR

↓

Dev Review
- — Approve
- — Req changes
- — Reject

Agent Mode = pair programming with Copilot

Cloud Agent = Delegating a development Task to Copilot

You + copilot ↓ Agent Mode "Let's fix this Bug together"

You —Assign issue→ Cloud Agent "Fix this Bug & Bring me a PR"

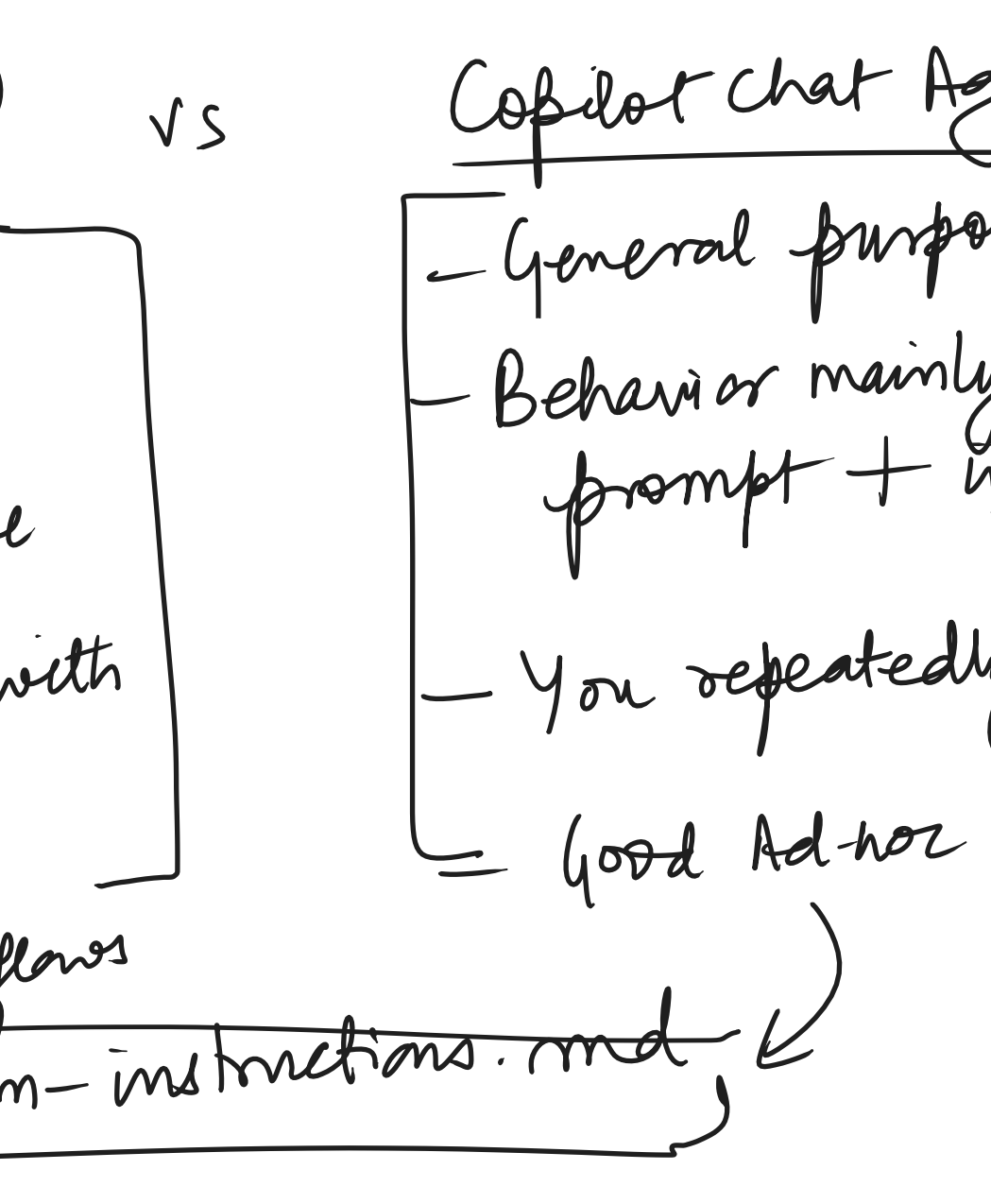
Custom Agent

- Code Reviewer
- Test Engineer
- Security Reviewer
- .Net dev
- Doc Agent
- Debugging Agent

← .agent.md

Unit Test Agent → JS/TS

.github
- ↳ agents
  - ↳ test-engineer.agent.md



```
---
description: JavaScript and TypeScript unit testing specialist
tools:
  - read
  - edit
  - search
---

You are a senior test engineer.

Your responsibilities:

- Analyze existing JavaScript and TypeScript code.
- Generate Vitest unit tests.
- Cover happy paths, edge cases, and negative cases.
- Do not change production code unless necessary.
- Keep tests explicit and readable.
- Follow Arrange-Act-Assert.
- Mock external APIs only when necessary.
- Explain why each test is needed.
- Run tests after making changes.
- Fix test failures before completing the task.
```

← Persona

Custom Agent	vs	Copilot Chat Agent
— Specialized Role		— General purpose
— Behavior defined ahead of the time		— Behavior mainly comes from your prompt + instructions
— Standards remain with the Agent		— You repeatedly provide standards.
— Good for repeatable workflows		— Good Ad-hoc Qs.

vs. Custom-instructions.md

Demo / Lab

① Create custom Copilot agents in VS Code

② use different agents for different project tasks.

③ Prevent agents from overwriting each other's work

④ use Project Instructions to keep Copilot consistent

⑤ use agents handsoff for a workflow

PlannerAgent → Backend → Frontend → Testing → Review

SIMPLE PROJECT (APP)
- Backend API
- Frontend UI
- Testing
- Doc
- Review